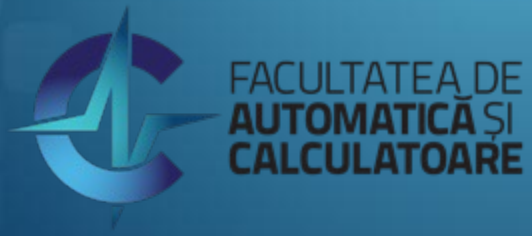


Algoritmi Paraleli și Distribuiți
Toleranta la defecte

Radu-Ioan Ciobanu
radu.ciobanu@upb.ro





Obiectiv

- Asigurarea tolerantei la defecte
  - Solutii de detectie a erorilor in SD
  - Solutii de recuperare din defect in SD

"You know you have one when the crash of a computer you've never heard of stops you from getting any work done."

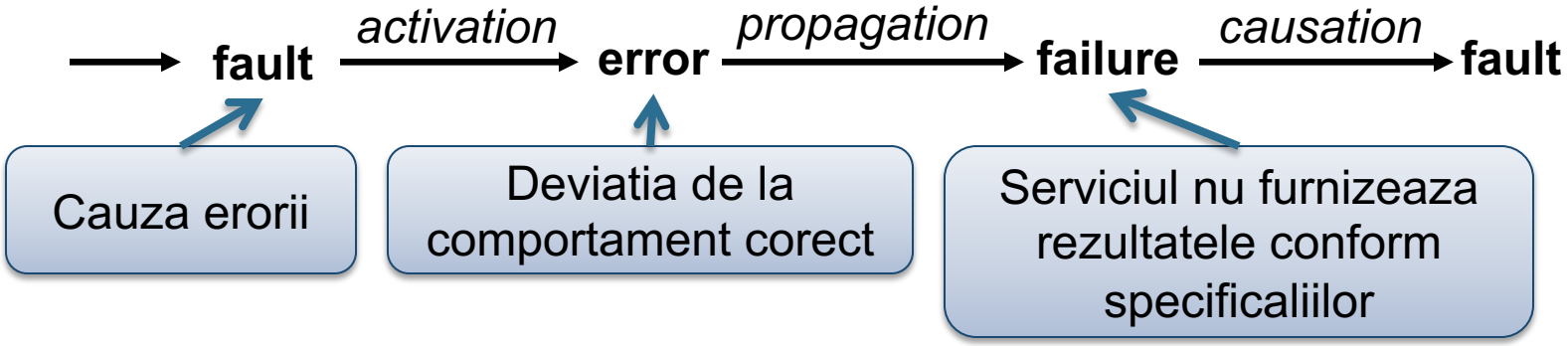
Leslie Lamport, 1993 – defectul in SD

Introducere

- Un sistem distribuit (SD) se defineste ca avand 6 caracteristici importante [Coulouris, et al, 1994]
  - Suportul pentru partajarea resurselor
  - Deschidere
  - Concurenta
  - Scalabilitate
  - Toleranta la defecte
  - Transparenta
- **Problema:** funcţionarea corectă în prezenţa unor defecţiuni ale sistemului
- Sistemele distribuite sunt compuse din multe componente hardware si software => probabilitate mai mare de aparitie a unor defecte in diverse parti

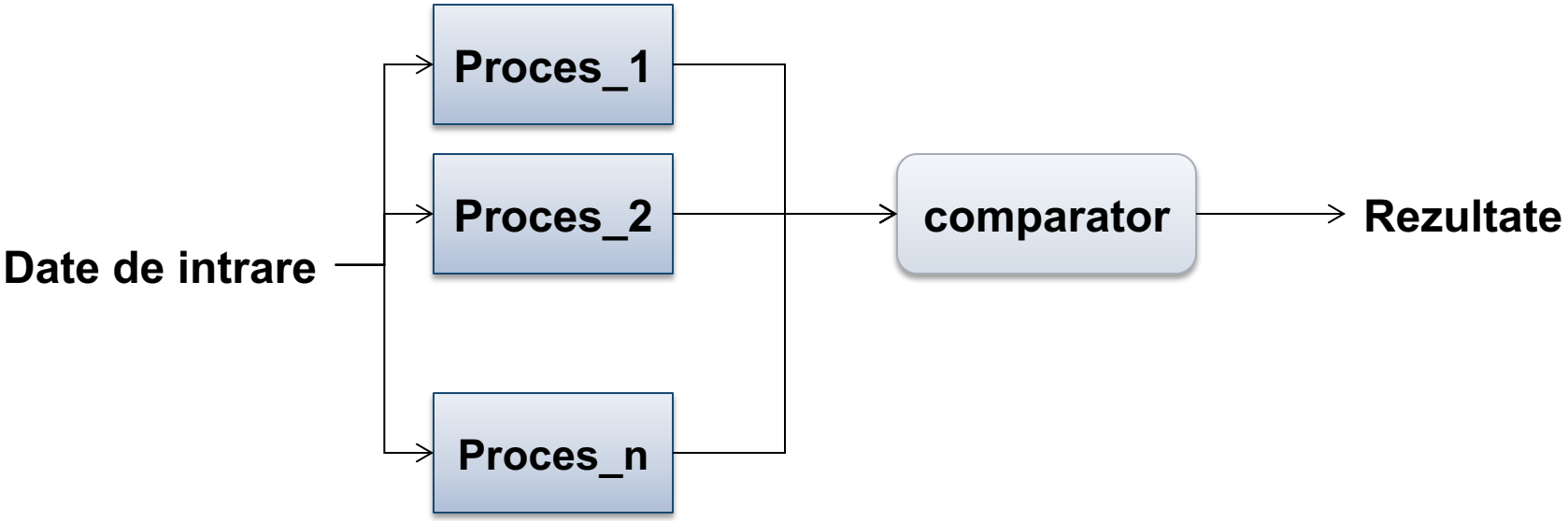
Toleranta la defecte

- Problemele: greseli de functionare, erori, defecte



- **Toleranta la defecte**:
  - Evitarea greselilor in functionare chiar in prezenta defectelor
  - Utillizatorii nu resimt defectarea unor componente.
- Doua abordari :
  - Redundanta Hardware
  - Recuperarea Software

Toleranta la defecte

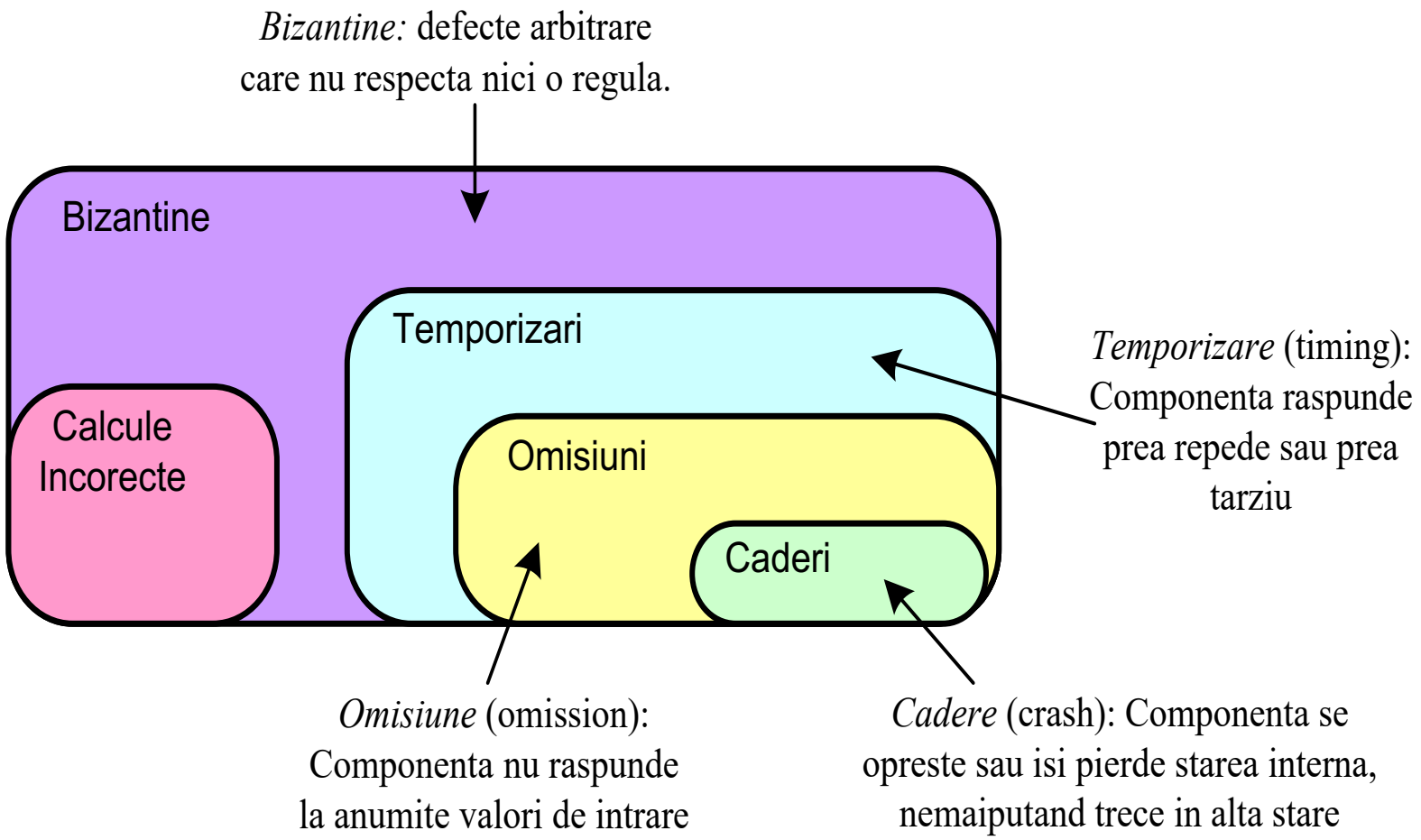


- Aceeaşi ieşire pentru aceleaşi valori de intrare,
- Valori apropiate la ieşire, pentru valori de intrare apropiate

Tipuri de defecte în sistemele distribuite

- Crash: la un moment dat un procesor devine nefuncţional
  - În ultimul pas, înainte de a se defecta, se poate să fi trimis numai un subset din mesaje
- Byzantine: procesorul intră în stări arbitrare şi trimite mesaje conţinând conţinut arbitrar

Clasificarea defectelor si anomaliilor



Toleranta la defecte in sistemele distribuite

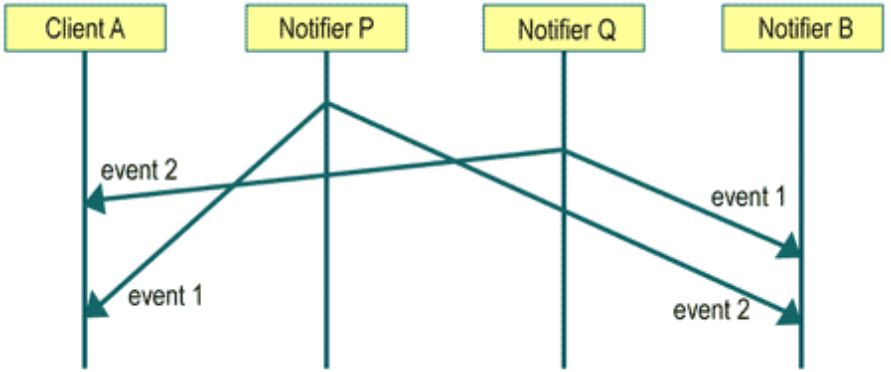
- Exemple:

COMPONENTA	DEFECT
Unitate de procesare (CPU)	Caderi sau defecte bizantine
Reteaua de comunicatie	Toate tipurile de defecte
Ceasurile	Temporizari, omisiuni, defecte bizantine
Unitate de stocare (disc)	Temporizari, omisiuni, caderi
Software	Toate tipurile de defecte

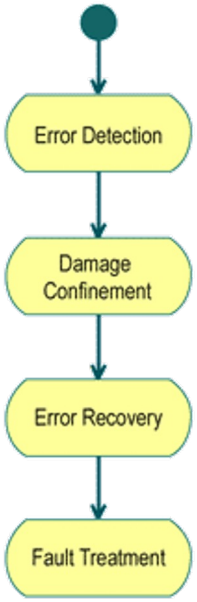
- Din perspectiva tolerantei la defecte, sistemele distribuite au un avantaj major: *se poate introduce usor o redundanta in sistem* deoarece imperfectiunile si posibilitatea unor defecte fizice nu pot fi ignorate.
- Probleme:
  - Defecte la nivelul nodurilor din sistemul distribuit
  - Defecte la nivelul procesului de comunicatie
  - Intarziere in procesul de transmisie a informatiei

Toleranta la defecte in sistemele distribuite (2)

- Defecte la nivelul nodurilor din sistemul distribuit
  - Defecte independente si diferite pe fiecare nod
- Defecte la nivelul procesului de comunicatie
  - *Defectiune hardware* a mediului de transmisie -> oprirea comunicatiei, partitionarea sistemului in mai multe sisteme
  - *Intermittent failure*: mesajele pierdute, schimbate ordinea sau duplicate
- Intarziere in procesul de transmisie al informatiei
  - Nu este un defect, dar poate conduce la defecte
  - Doua situatii frecvent intalnite:
    - *Jitter (variable delay)* -> timpul in care un mesaj ajunge la destinatie
    - *Efectul relativistic* -> site-uri diferite pot primi aceleasi mesaje, dar in alta ordine

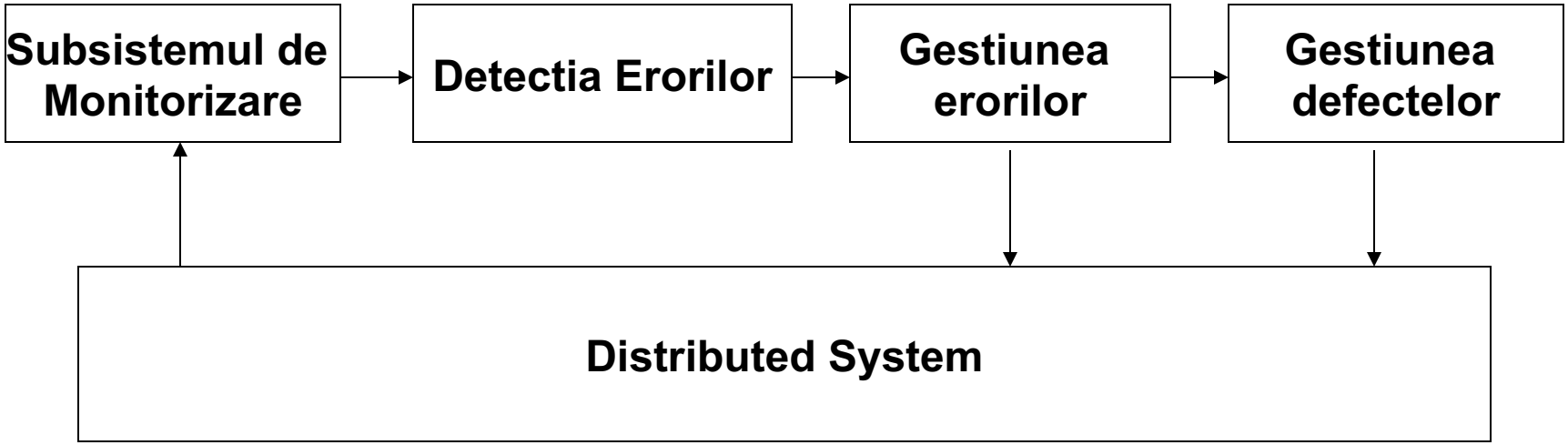


Fazele tolerantei la defecte



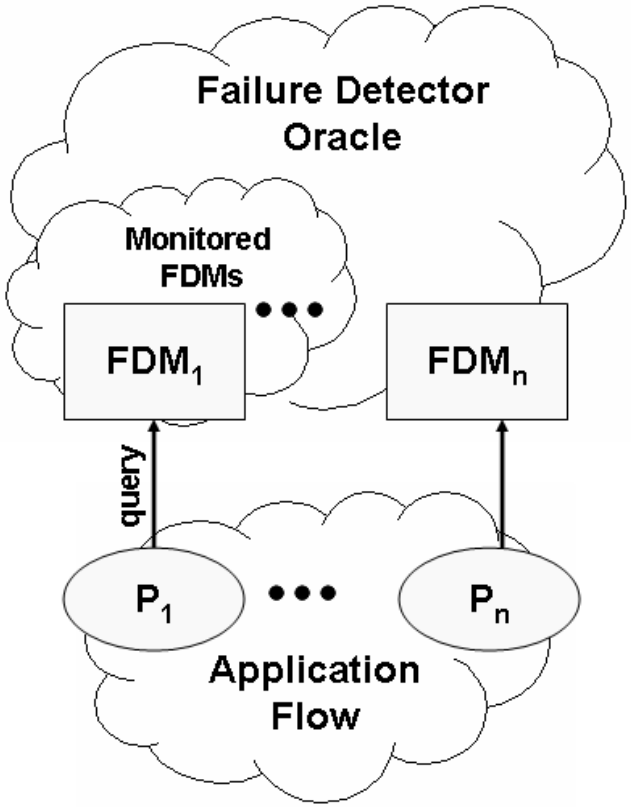
- Proiectarea si implementarea unui sistem distribuit tolerant la defecte este dependenta de arhitectura sistemului si de functionalitatea sa. Prin urmare nu exista reguli generale pentru a adauga caracteristici de toleranta la defecte intr-un sistem, insa putem formula unele principii care trebuie sa urmeze patru faze:
  - Detectia erorilor
  - Estimarea si izolarea zonelor afectate
  - Recuperarea erorilor
  - Tratarea defectelor.

Arhitectura generica de FT



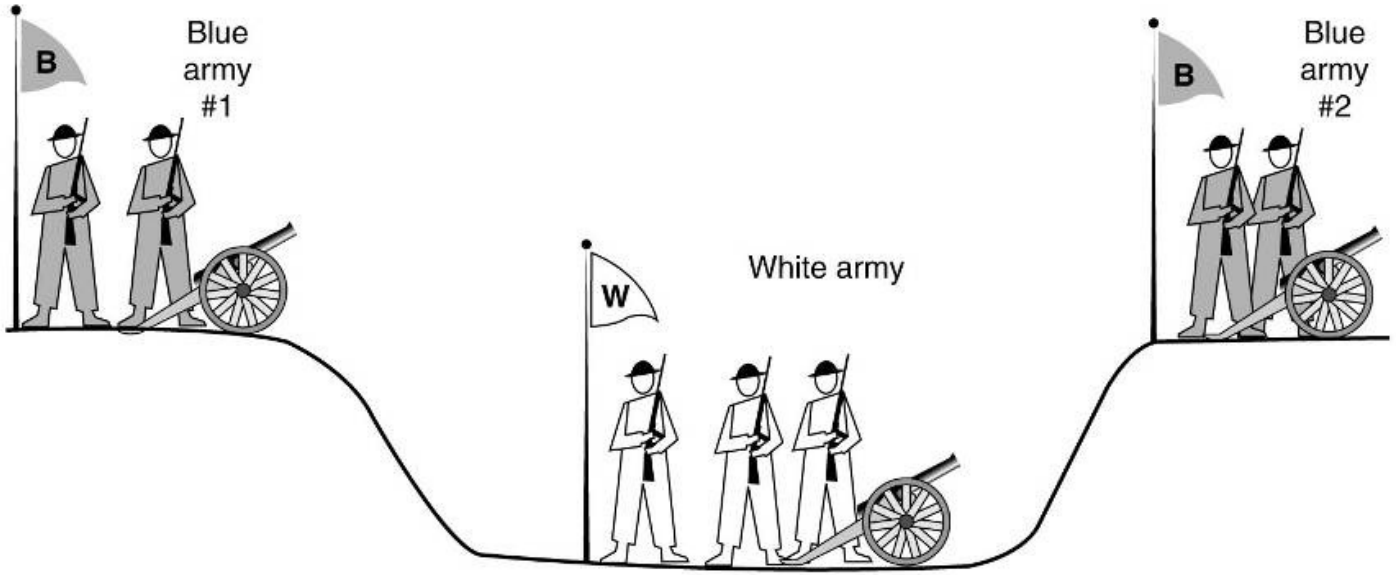
Detectoare

- Nivelul de FD este in general asigurat de detectoare locale dedicate



Problema celor două armate

- Puteți să vă gândiți la un algoritm?



Scheme de detectie

- Strategia Heartbeat – traditionala, (inca) cea mai intalnita
- Problema – scalabilitatea
  - Solutia 1) Structuri ierarhice de FD – Two-level Globus FD
  - Solutia 2) Protocoale de tip Gossip – cu o mare probabilitate, in cele din urma toate procesele obtin aceleasi informatii
- Protocoale Adaptive
  - Se adapteaza dinamic la conditiile de trafic peste retea
  - E.g.: Un protocol ce adapteaza valoarea intervalului de timeout in functie de valoarea maxima a timpului de raspuns al mesajelor de heartbeat
- Detectoare de erori lenese
- Detectoare de erori progresive

Protocoale de detectie adaptive

- Asigura in aceeasi masura adaptarea la:
  - schimbarile conditiilor de retea (conditii diverse de trafic)
  - cerintele si schimbarile de context ale aplicatiilor
- La baza tot strategia *heartbeat,* insa intervalul de timeout este modificat dinamic conform conditiilor de retea

Exemple FD adaptive (1)

- FETZER-FD
  - Intervalul de timeout este calculat ca fiind maximul dintre timpii de receptionare a mesajelor heartbeat
  - In conditii reale:

1. in retele mari probabilitatea de pierdere a mesajelor este ridicata
2. nu exista o limita superioara in ceea ce priveste intarzierile de mesaje

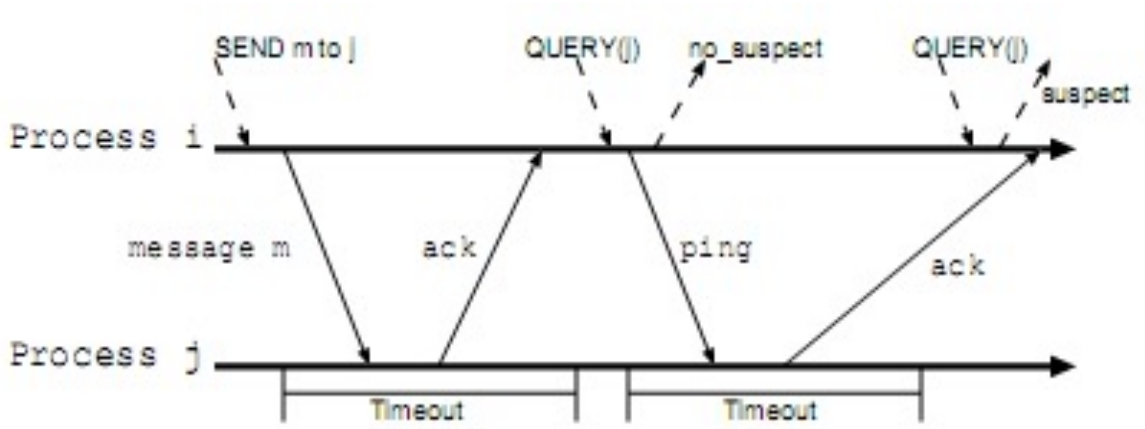
Exemple FD adaptive (2)

- CHEN-FD
  - Se bazeaza pe analiza probabilistica a traficului de retea si a cerintelor aplicatiilor
  - Estimarea timpului de primire a urmatorului mesaj heartbeat pe baza timpilor de receptionare a celor mai recente mesaje heartbeat
  - Estimarea anterioara plus o margine de siguranta formeaza intervalul de timeout

Dezavantaje

- Acuratetea este data de functia de estimare
- Adaptarea la cerintele mai multor aplicatii ar necesita administrarea mai multor valori de timeout
- O solutie alternativa - aplicatiile sa-si seteze propria valoare de timeout
  - dar se invalideaza adaptarea la conditiile de retea

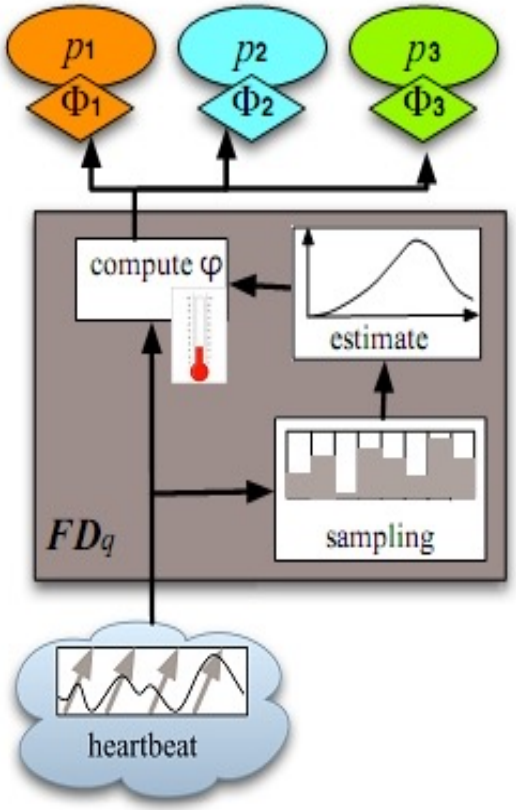
Detectoare de erori lenese



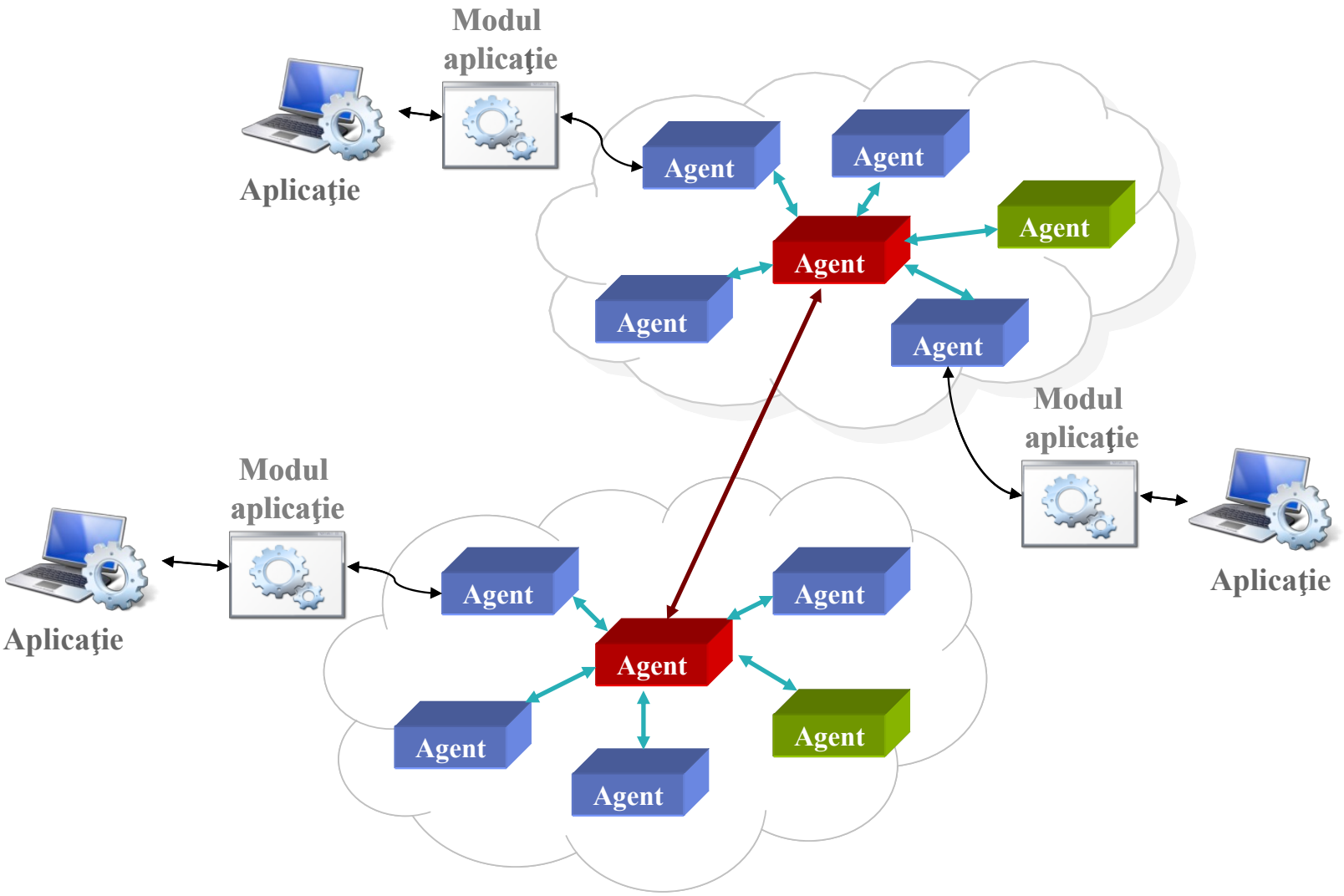
- Ideea de baza este folosirea a cat mai multe mesaje transmise intre procese pentru a detecta erori → se impune, astfel, ca fiecare mesaj primit sa fie confirmat
- Cand procesele nu comunica, se folosesc mesaje de monitorizare

Detectoare de erori progresive

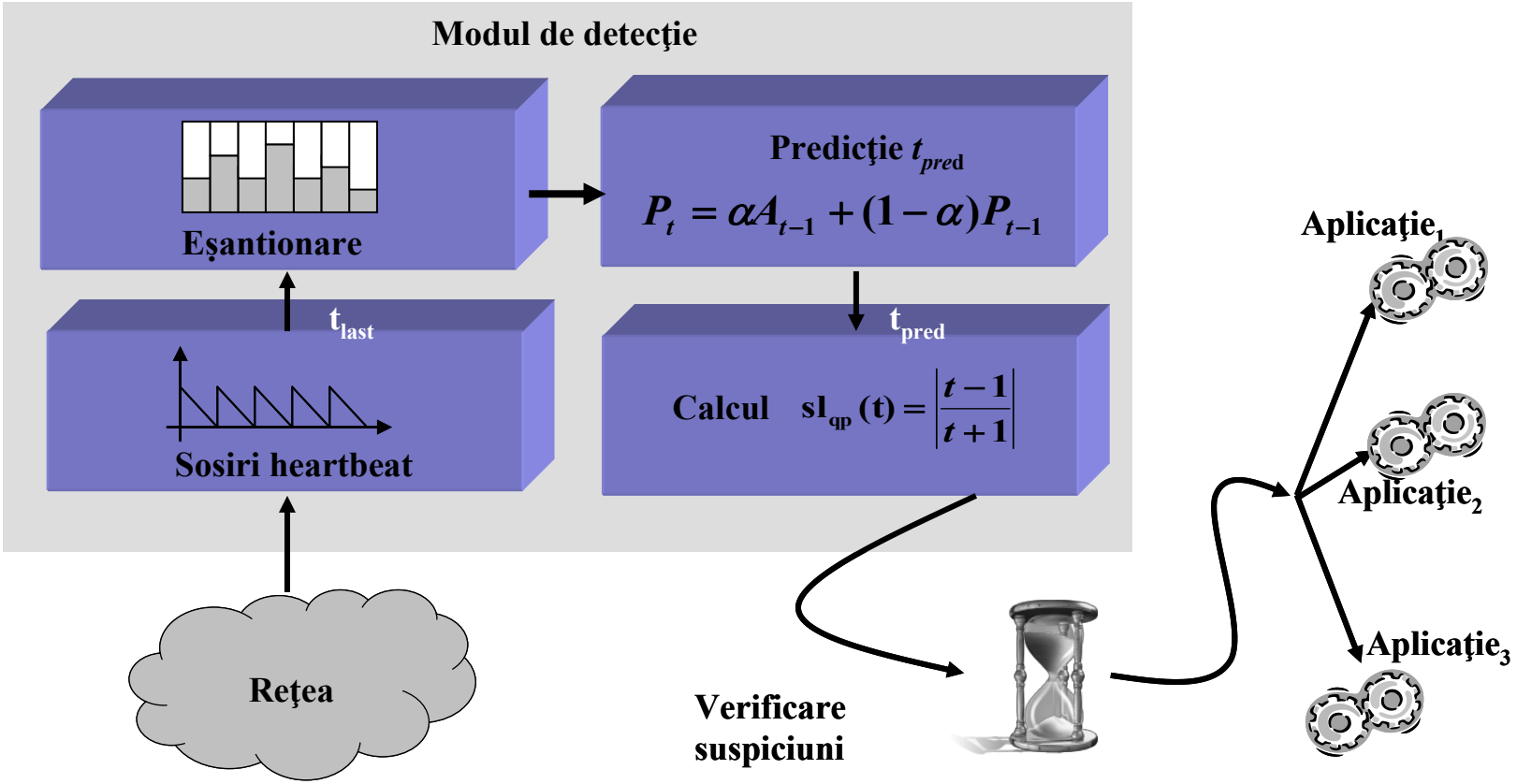
- Se bazeaza pe asocierea unui proces cu o valoare progresiva, care creste monoton in timp daca procesul s-a defectat
- Fiecare aplicatie are un prag finit stabilit conform propriilor cerinte
- O aplicatie decide daca suspecteaza sau nu un proces, comparand pragul propriu cu valoarea progresiva asociata procesului



Clusterizare



Interpretare date monitorizare (1)



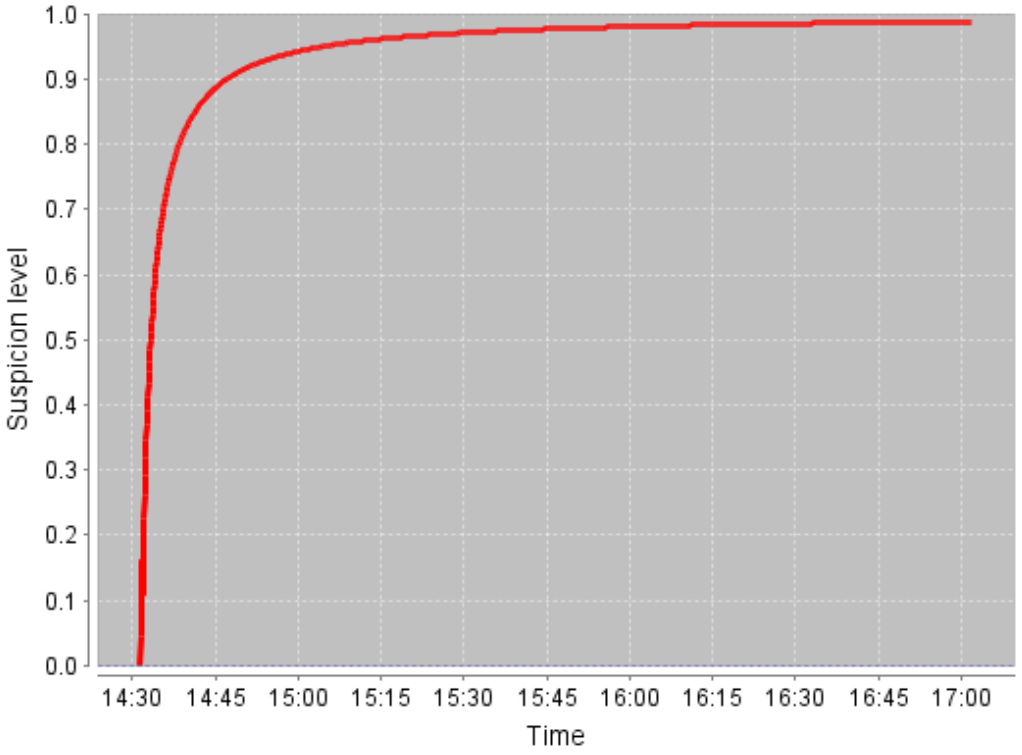
Interpretare date monitorizare (2)

$$0 <= sl_{qp}(t) < 1 \mapsto sl_{qp}(t) = \left|\frac{t-1}{t+1}\right|,$$

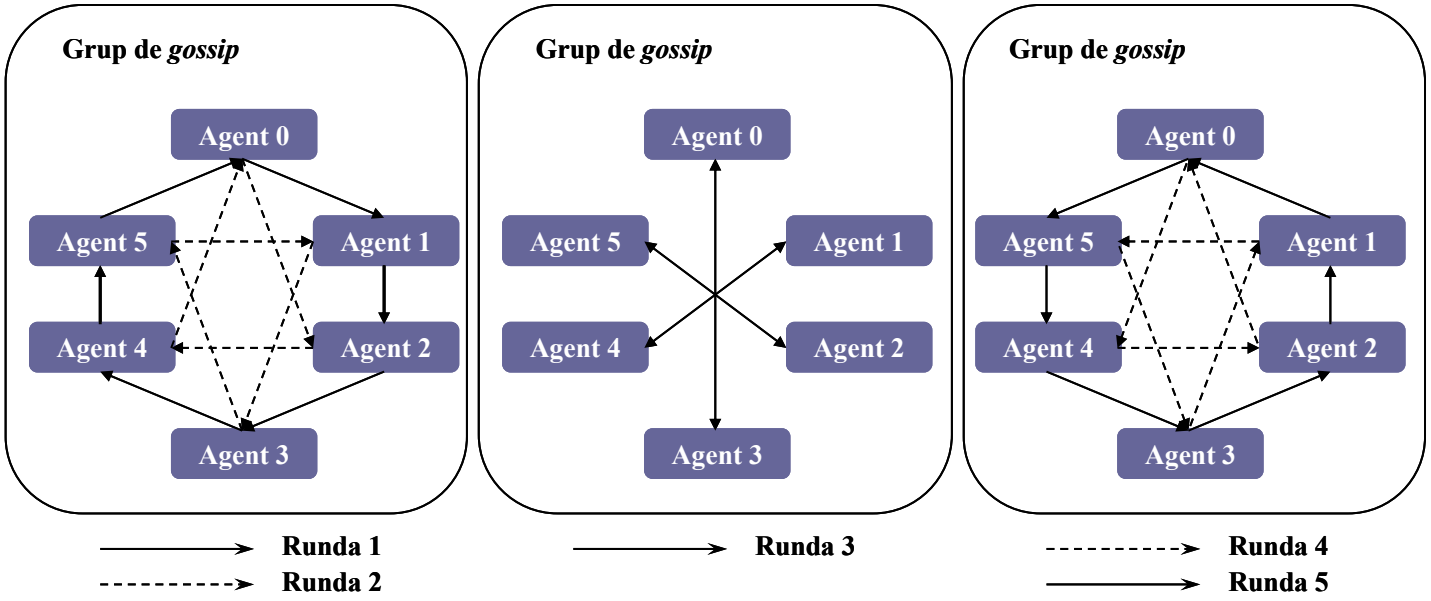
$$t = \frac{t_{now}}{t_{pred}} \in (0, \infty),$$

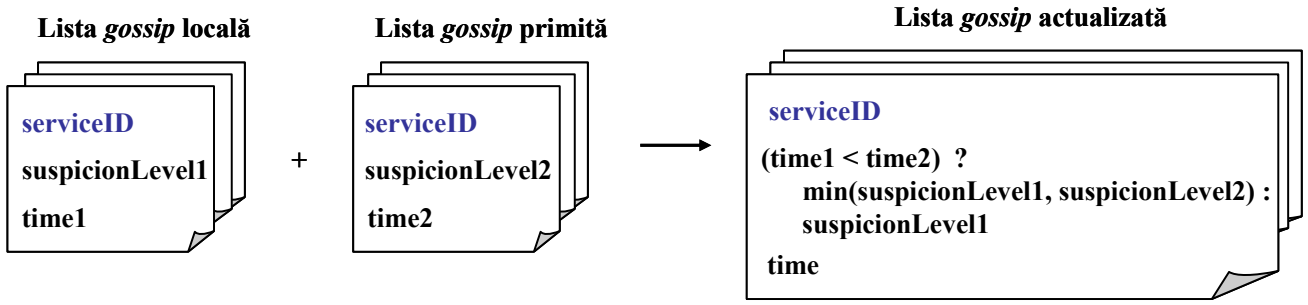
t_{now} *timpul curent*,

t_{pred} *ultima valoare prezisa*

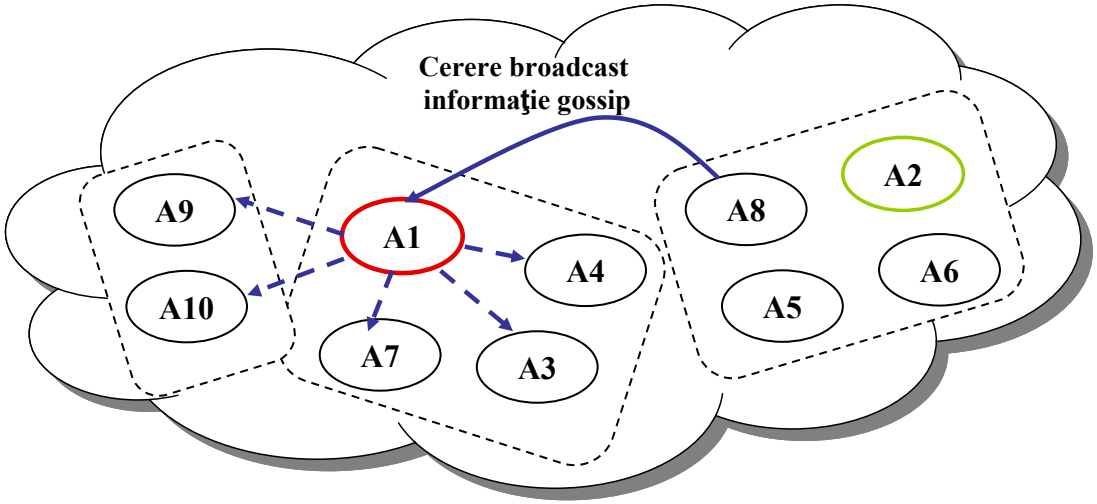


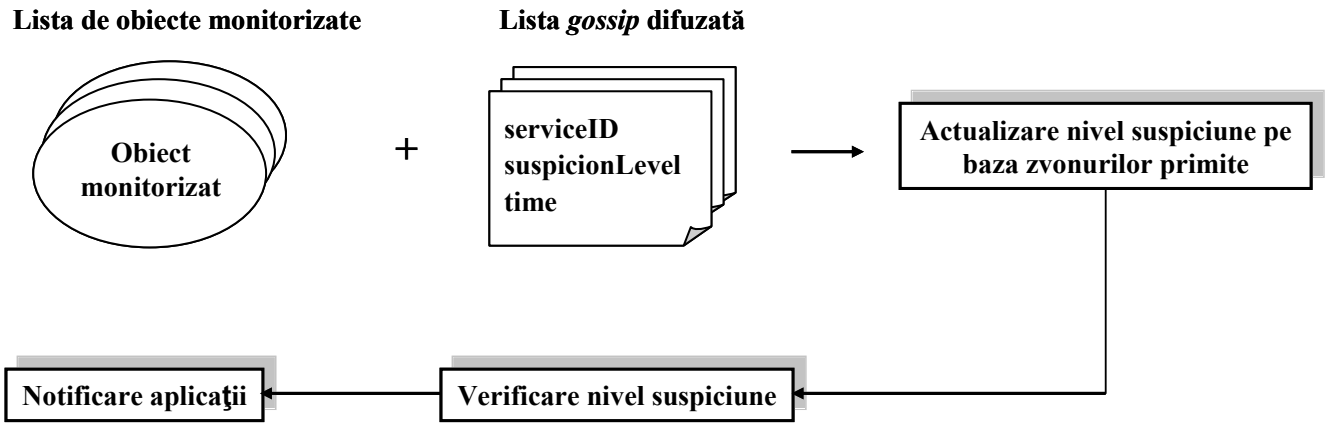
Gossiping (1)


Grup de *gossip*
Agent 0
Agent 1
Agent 2
Agent 3
Agent 4
Agent 5
Grup de *gossip*
Agent 0
Agent 1
Agent 2
Agent 3
Agent 4
Agent 5
Grup de *gossip*
Agent 0
Agent 1
Agent 2
Agent 3
Agent 4
Agent 5
Runda 1
Runda 2
Runda 3
Runda 4
Runda 5


Lista *gossip* locală
serviceID
suspicionLevel1
time1
+
Lista *gossip* primită
serviceID
suspicionLevel2
time2
Lista *gossip* actualizată
serviceID
(time1 < time2) ?
min(suspicionLevel1, suspicionLevel2) :
suspicionLevel1
time

Gossiping (2)


Cerere broadcast
informaţie gossip
A9
A1
A4
A8
A2
A10
A7
A3
A5
A6

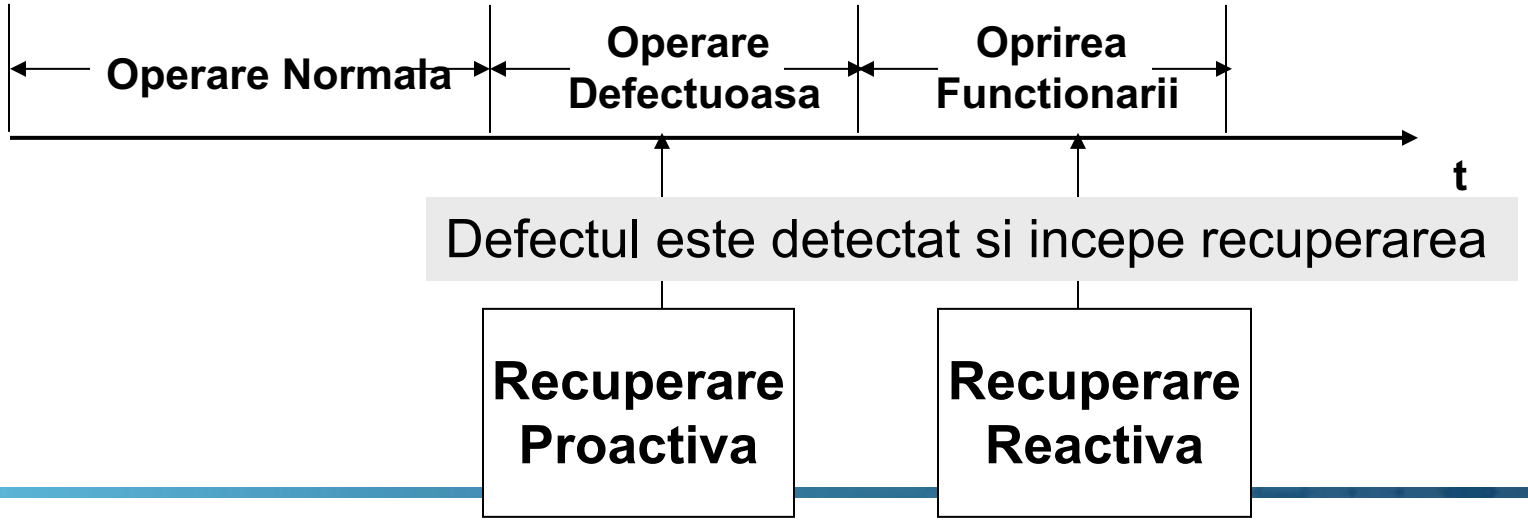

Lista de obiecte monitorizate
Lista *gossip* difuzată
Obiect monitorizat
+
serviceID
suspicionLevel
time
Actualizare nivel suspiciune pe baza zvonurilor primite
Notificare aplicaţii
Verificare nivel suspiciune

Recuperarea din eroare

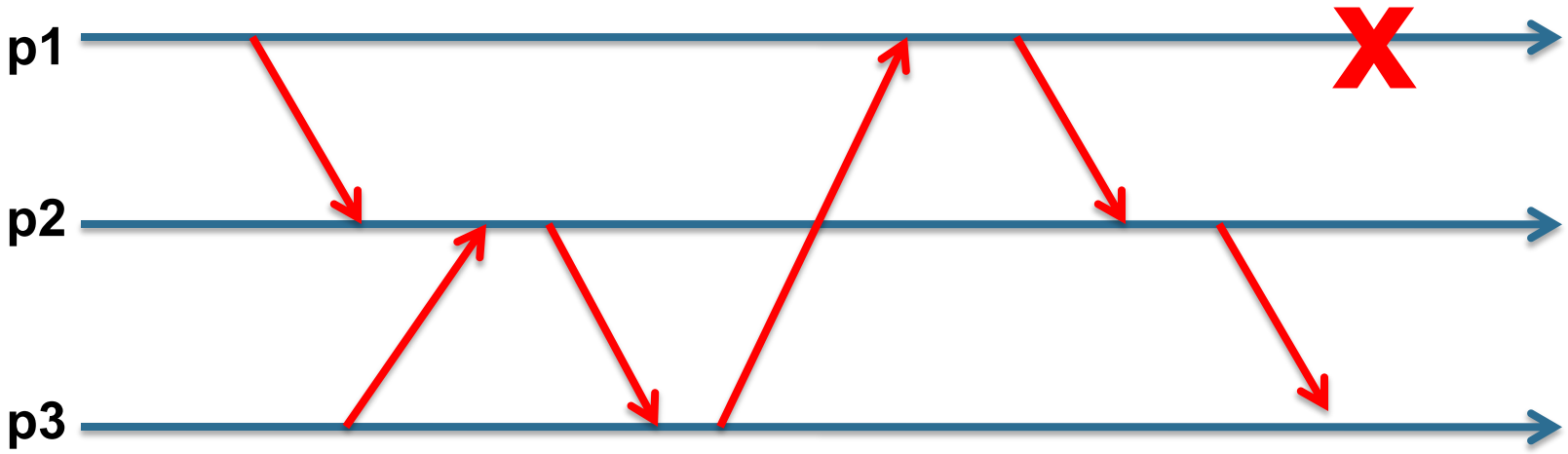
- Realizata prin “readucerea” aplicatiei la o stare corecta, de dinainte de aparitia defectului
- Gestiunea erorilor:
  - Rollback – stare salvata: checkpoint
  - Rollforward – urmatoarea stare fara eroare reprezinta noua stare
  - Compensare – mascarea erorii prin redundanta

Solutii de recuperare din eroare

- Abordarea tipica FT: sistemul asteapta aparitia unui defect si reactioneaza la detectarea acestuia pentru a-l corecta
- Daca defectele se propaga mai repede decat pot mecanismele de recuperare sa le trateze se poate ajunge la situatia cand nu se mai pot aplica tehnicile de recuperare
- Mecanismele de recuperare proactiva:
  - Cresterea rezistentei sistemului la posibilele propagari nedorite ale erorilor
  - Initierea recuperarii inainte sau concurent cu recunoasterea problemei din sistem
  - Adaptarea sistemului

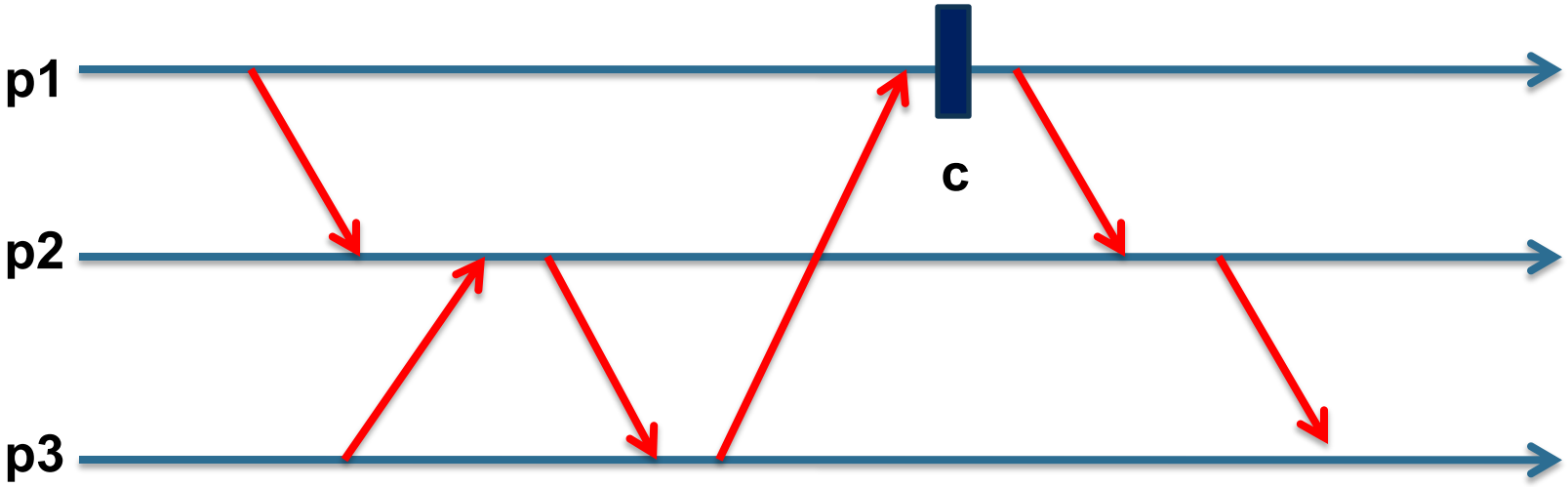


FT pentru aplicatiile bazate pe schimburi de mesaje



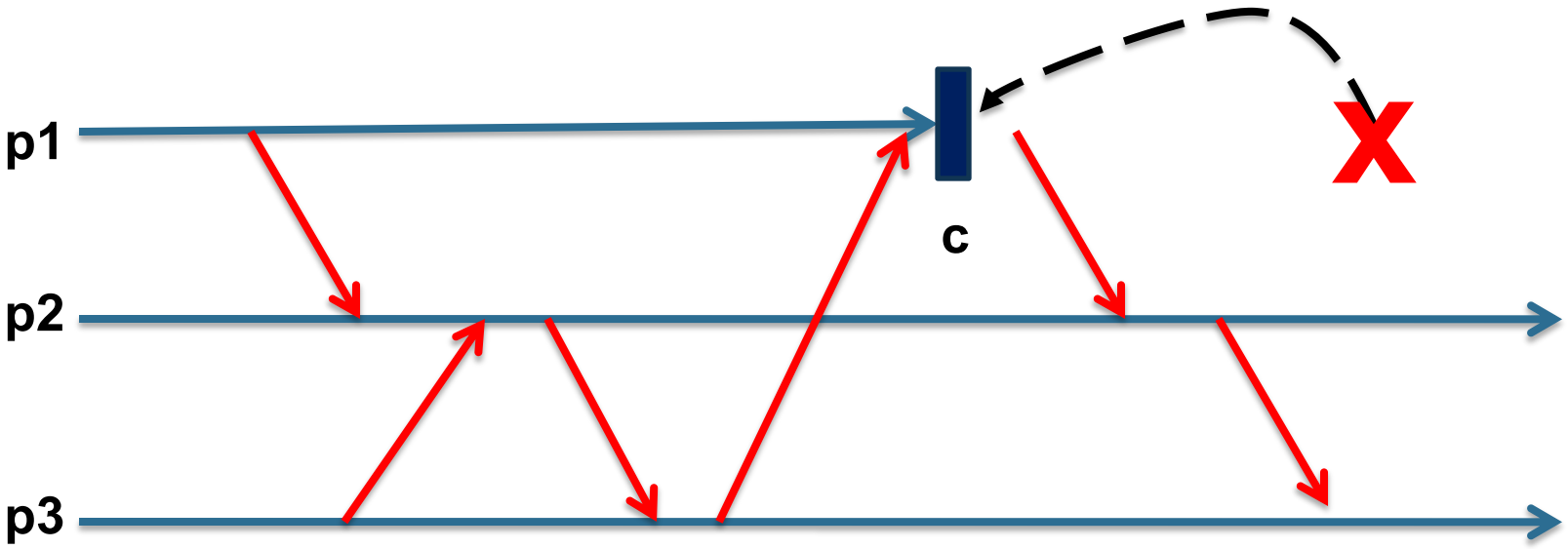
- Re-executia intregii aplicatii
- Pierderea tuturor calculelor

Checkpoint



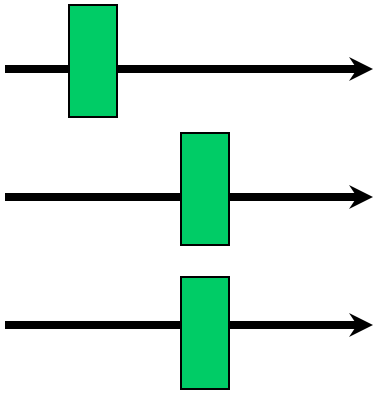
- Snapshot al unui proces
- Starea salvata pe medii de stocare persistente

Checkpoint



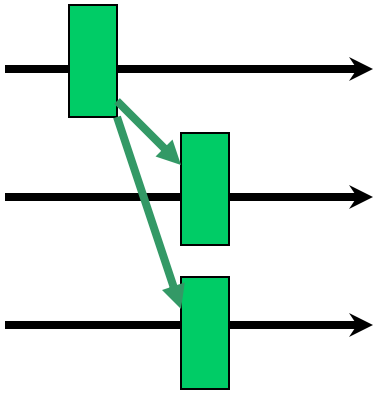
- p2 a trimis un mesaj ce nu a fost trimis de p1
- Este nevoie de salvea unei stari globale, consistente, a sistemului

Checkpointing Distribuit



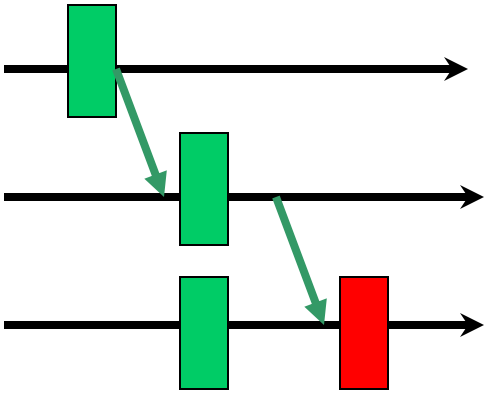
Independent

- ☑ Simplitate
- ☑ Autonomie
- ☑ Scalabilitate
- ✖ Efectul de Domino



Coordonat

- ☑ Stari Consistente
- ☑ Performanta Buna
- ☑ Garbage Collection
- ✖ Scalabilitate

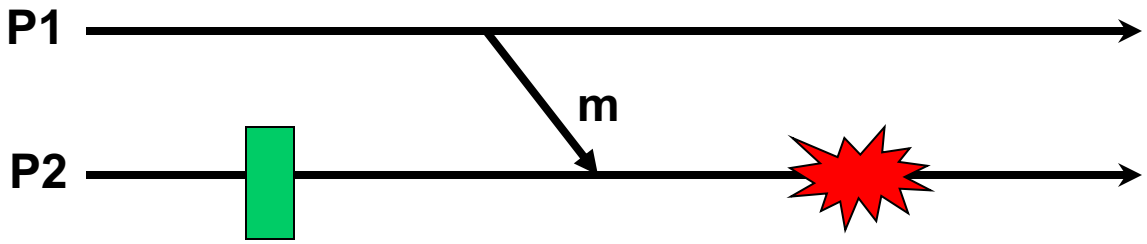


Indus de comunicatie

- ☑ Stari Consistente
- ☑ Autonomie
- ☑ Scalabilitate
- ✖ Performanta

Logarea Mesajelor

- Logarea datelor pe dispozitive persistente de stocare in timpul executiei fara erori
- Folosirea acestei informatii pentru recuperarea din eroare

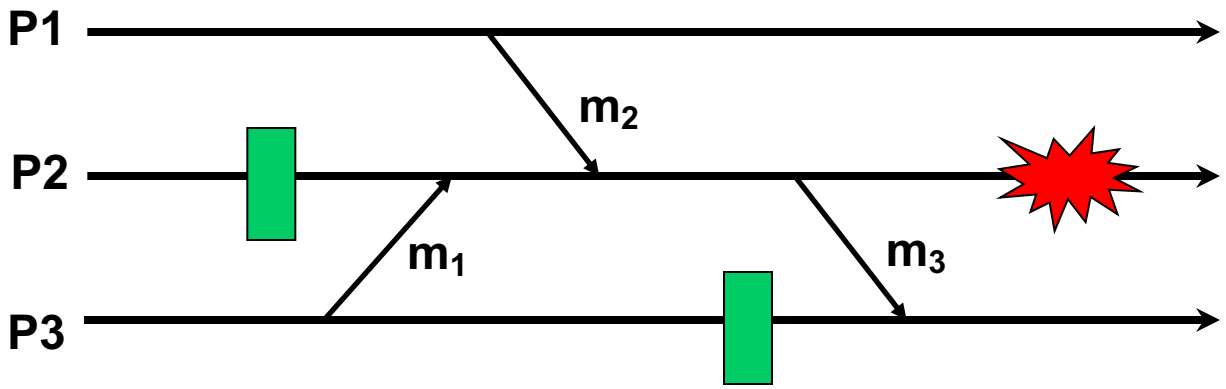


- ***Orfan:*** procesul ce depinde de o stare ne-recuperabila a unui proces esuat

Procese orfan?

- Dupa ce procesul p trimite mesajul m, tot el trimite mesajul m' catre procesul q, care la randul lui il trimite mai departe.
- Daca mesajul m nu era logat, atunci defectarea lui p ar putea conduce la pierderea oricarei informatii despre m.
- La reinitializarea lui p, el poate ajunge intr-o stare in care a doua oara nu mai trimite mesajul m' lui q.
- In acest caz procesul q nu ar mai fi consistent cu procesul p, pentru ca q a trimis un mesaj m' ce nu a fost niciodata trimis de procesul p.
- q este un proces orfan.

Abordari Message Logging



Pesimista

☑Nu exista procese orfan

☑Recuperare usoara

✖Performante scazute

Optimista

☑ Non-blocanta

✖ Procese orfan

✖ Recuperare complexa

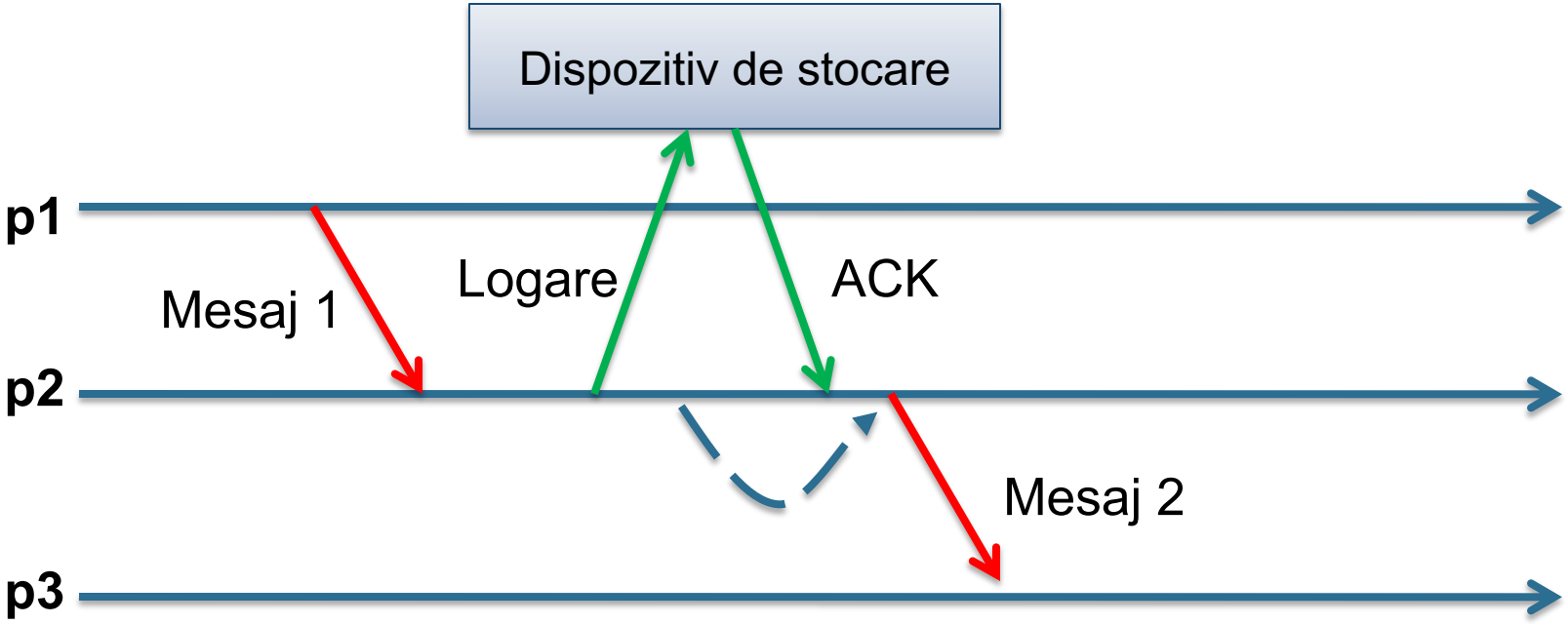
Cauzala

☑ Non-blocanta

☑ Nu exista procese orfan

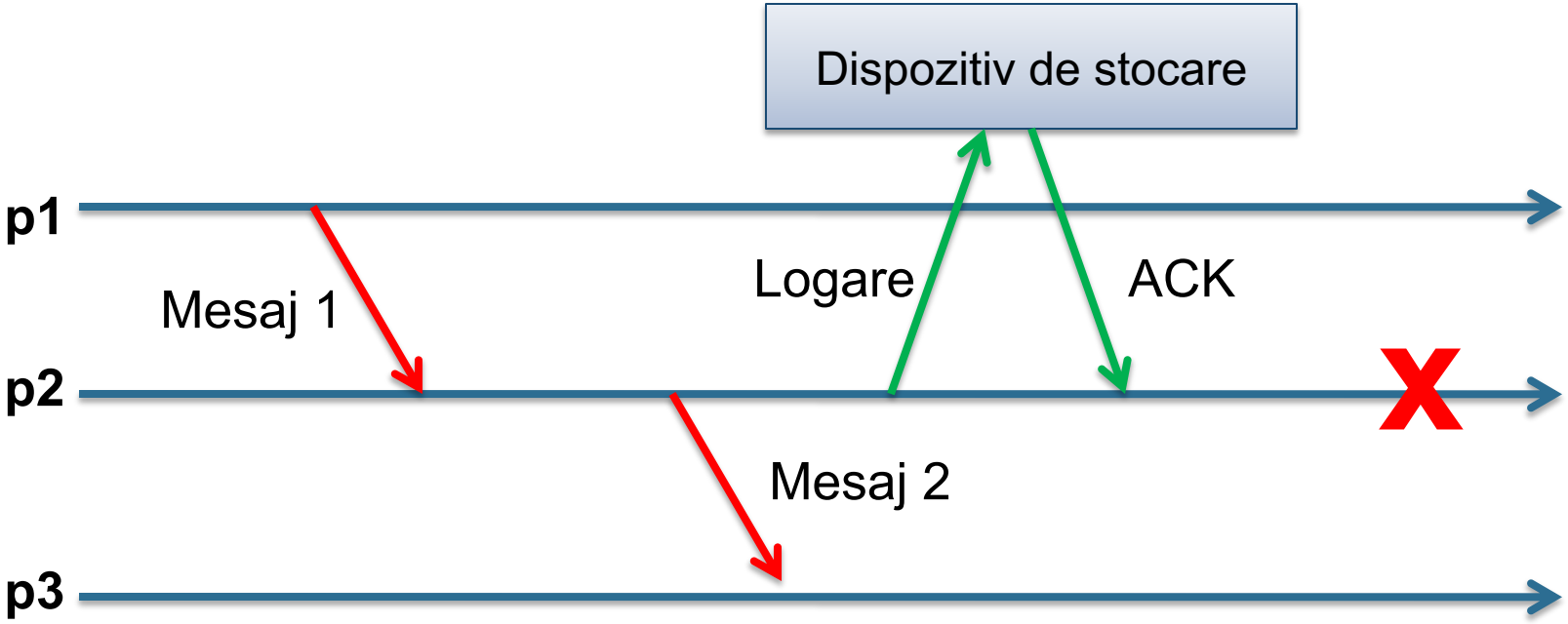
✖ Recupera Complexa

Abordarea pesimista



- Mesajele sunt logate sincron pe mediul de stocare
- Overhead asupra comunicarii

Abordare optimista



- Mesajele sunt logate asincron pe mediul de stocare
- Protocolul de recuperare poate fi complex (risc de aparitie a proceselor orfane)

Recovery Oriented Computing

- Ideea: defectele hardware, software si erorile de operare sunt aspecte ce trebui tratate, nu prevenite
- Axate pe MTTR:
  - Reducerea timpului de recuperare
  - Marirea disponibilitatii sistemului
- Repornirea sistemului dupa defect
  - Solutia naiva: reboot
  - Repornirea doar a unor parti ale sistemului: sisteme modulare, in care componentele pot fi repornite independent: serviciul din Unix /etc/rc
- Stateful recovery
  - Recuperarea din defect in cazul bazelor de date
  - Folosirea metodelor de checkpointing